

Tema V. Exemples de Llenguatges de Programació: LISP

21721 – Llenguatges de Programació

Antoni Oliver

Aina M. Tur

Miquel A. Cabot



Universitat
de les Illes Balears

LISP

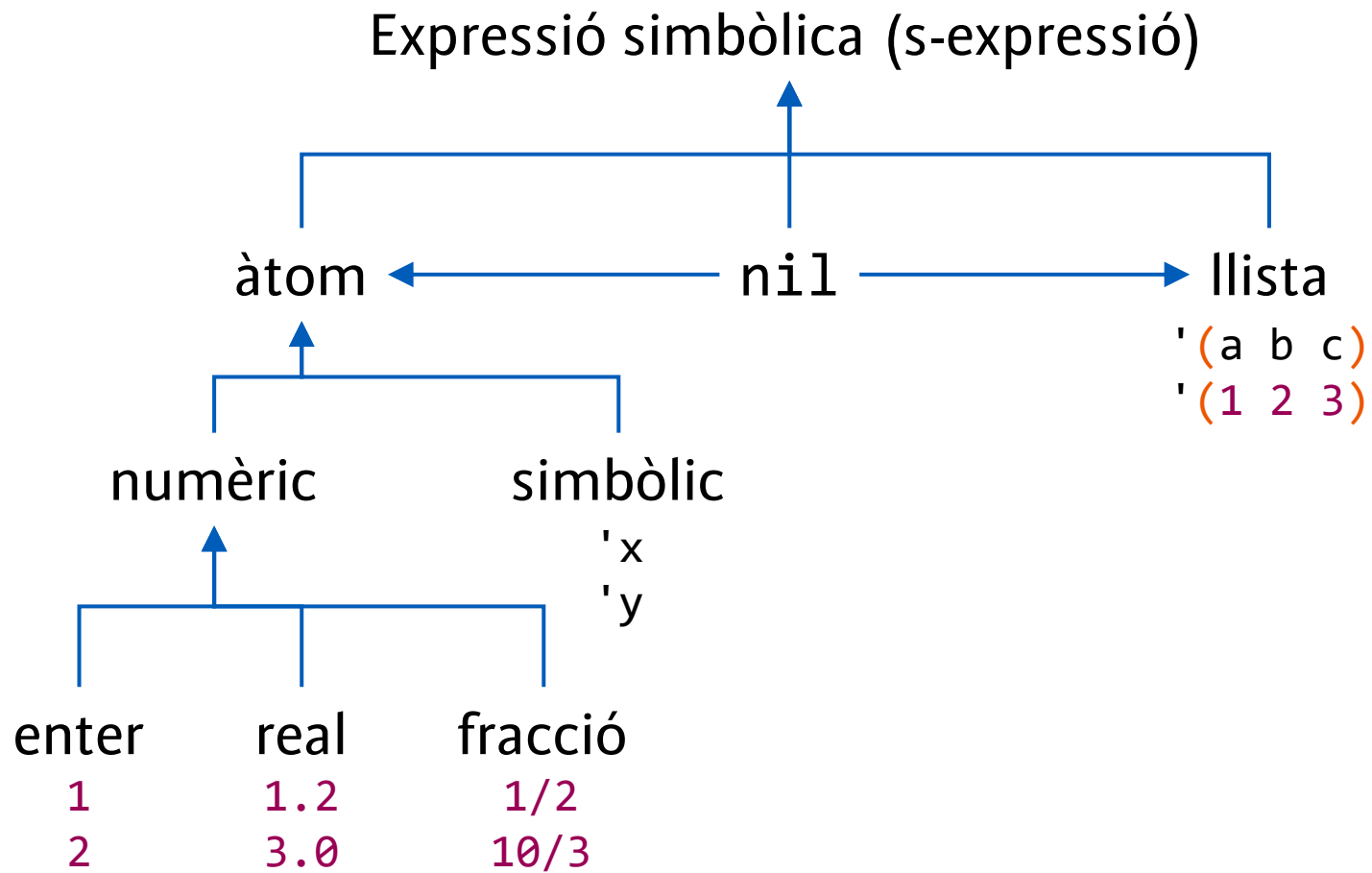
- ◆ Són les inicials de **LISt Processing language**
- ◆ Creat per John McCarthy, al MIT (1958)
- ◆ Dissenyat per la Intel·ligència Artificial
- ◆ Estudiarem la part funcional de LISP

LISP

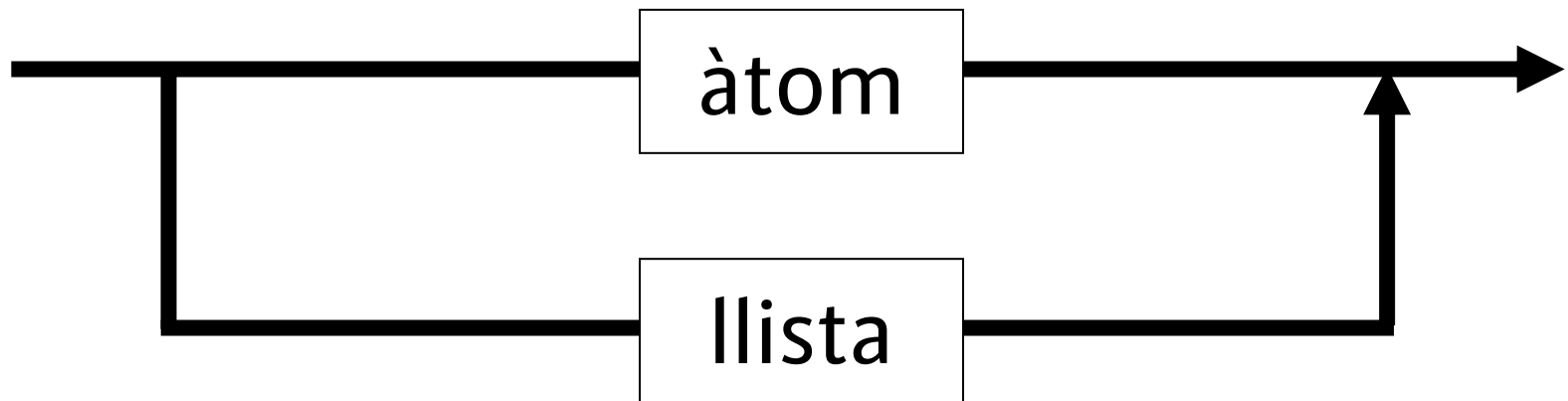
◆ LISP va introduir:

- ◆ L'equivalència entre estructures de dades i programes
- ◆ La **recursió** com a única estructura de control d'iteració
- ◆ La **llista** com a estructura de dades bàsica
- ◆ La recollida d'escombraries (*garbage collection*)

Tipus de dades bàsics



S-expressió



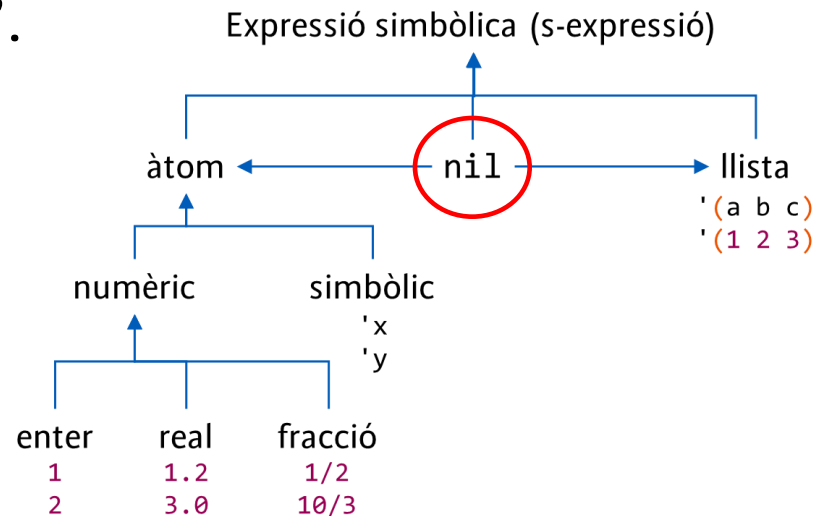
nil

◆ *nil*:

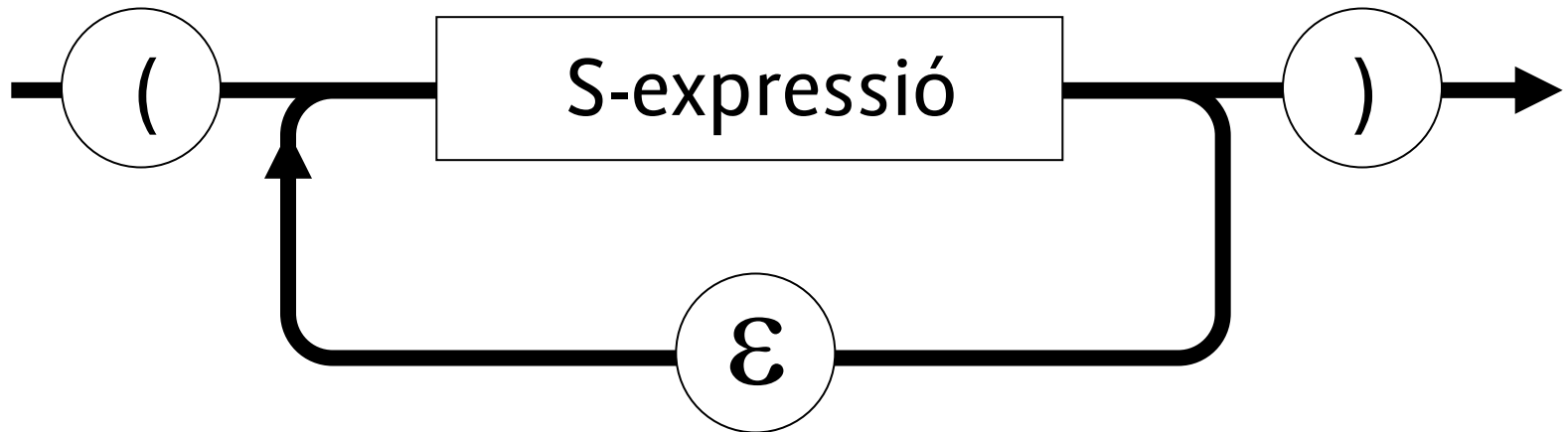
- ◆ El valor atòmic “fals”
- ◆ La llista buida (també representable amb '() o ())

◆ I el valor “cert”?

- ◆ Qualsevol valor no-nil és considerat “cert”.
- ◆ *t* sempre avalua a “cert”.



Llista

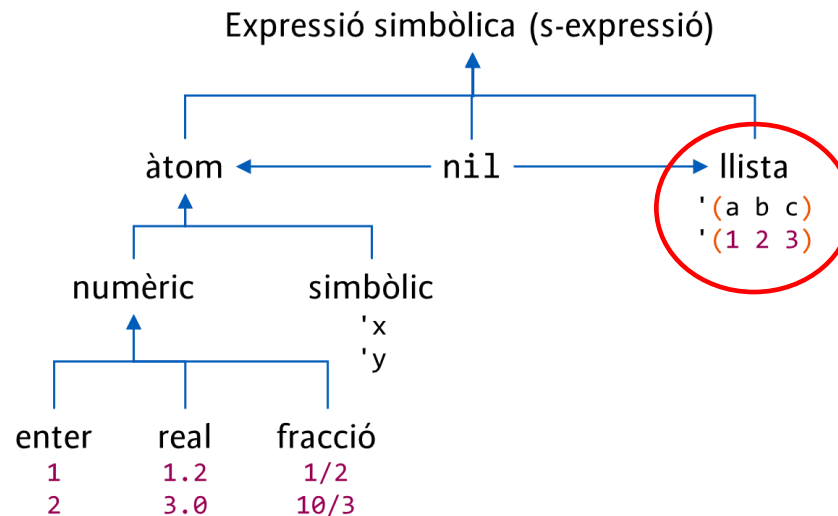
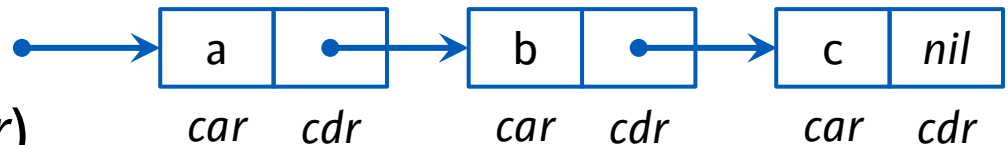


Llistes

◆ Llistes enllaçades:

◇ Valor del node (*car*)

◇ Enllaç al node següent (*cdr*)



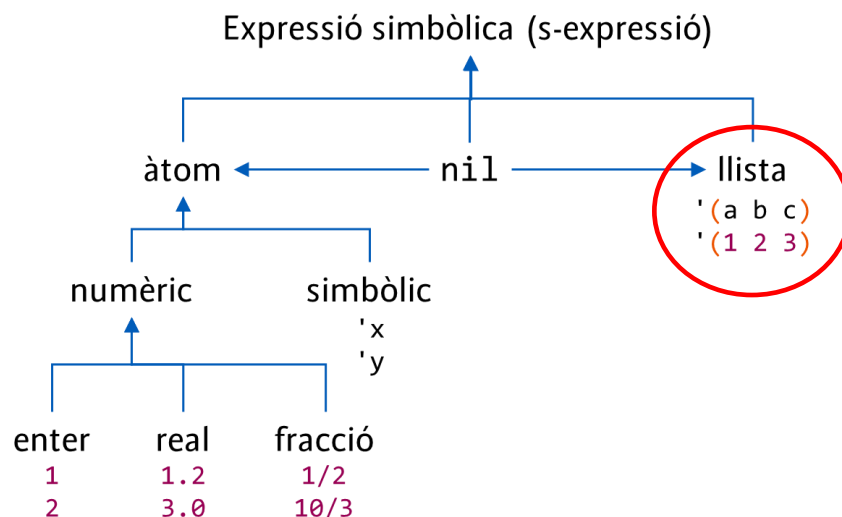
Llistes

◆ Exemples:

◆ (palma inca manacor)

◆ (a 35 b 44)

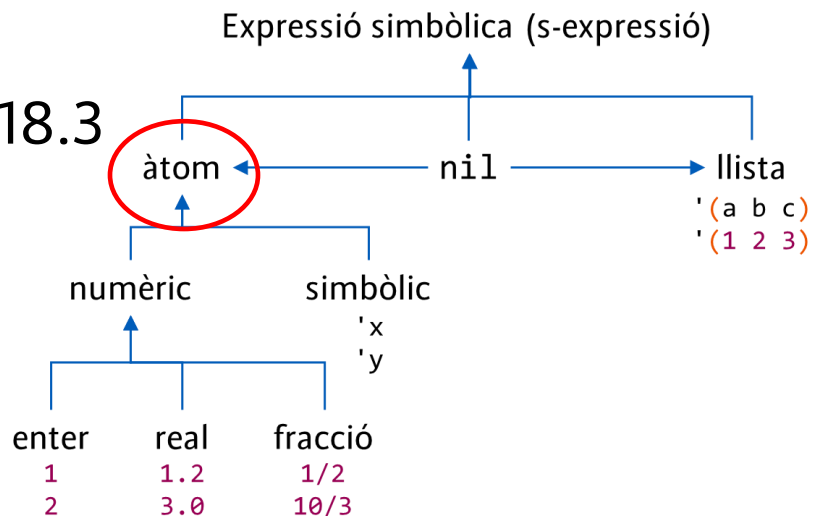
◆ ((antoni comes) (pep llopis))



Àtoms

- ◆ Els **àtoms simbòlics** (literals) són identificadors (lletres, números i guions començant per una lletra).
- ◆ Els **àtoms numèrics** estan formats únicament per xifres (i pot ser el punt decimal).
- ◆ Exemples:

◇ A Z80 PREU-COST 38 18.3



El procés d'avaluació

- ◆ Els sistemes de LISP són intèrprets que permeten a l'usuari introduir una expressió, l'intèrpret la llegeix, l'avalua i ens mostra el resultat.
 - ◇ REPL: read-eval-print-loop.
- ◆ Aquest procés d'avaluació es repeteix contínuament.

El procés d'avaluació

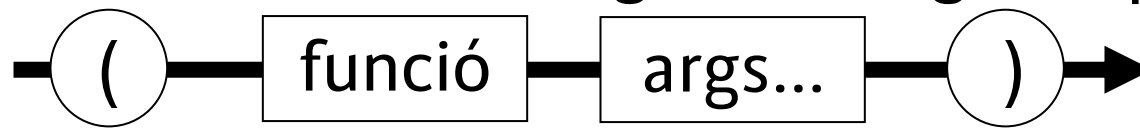
- ◆ Un àtom numèric avalua a ell mateix

- ◇ $30 \rightarrow 30$

- ◆ Un àtom simbòlic avalua al seu valor

- ◇ $\text{PI} \rightarrow 3.1415926$

- ◆ L'avaluació d'una llista segueix el següent procés:



- ◇ El primer element és el nom d'una funció.

- ◇ S'avaluen els arguments i després es crida la funció.

- ◇ $(+ 3 4) \rightarrow 7$

El procés d'avaluació

- ◆ Dos tipus de llistes: les dades i els programes, però les dues tenen el mateix format
 - ◇ (jo tu ell)
 - ◇ (+ 3 4)
- ◆ Per evitar que les llistes de dades s'avaluïn hem d'utilitzar l'operador QUOTE o ' (apòstrof).
 - ◇ ' (+ 3 4) o (quote (+ 3 4))
 - ◇ avaluen a (+ 3 4) i no a 7
- ◆ Feim el mateix amb els símbols:
 - ◇ ' nom per representar el símbol *nom*.
 - ◇ nom per avaluar el símbol *nom* (el seu valor).

El procés d'avaluació

- ◆ LISP utilitza la *Notació Prefixa de Cambridge*:
 - ◇ L'operador va al principi
 - ◇ Els operands a continuació separats per espais en blanc
- ◆ A què avalua $(+ 5 (* 1 4))$?

Operacions aritmètiques

◆ $(+ \ n_1 \ n_2 \ \dots \ n_N): n_1 + n_2 + \dots + n_N.$

◇ $(+ \ 1 \ 2) \rightarrow 3$

◇ $(+ \ 1 \ 2 \ 3) \rightarrow 6$

◆ $(- \ n_1 \ n_2 \ \dots \ n_N): n_1 - n_2 - \dots - n_N.$ Si $N=1$, $-n_1.$

◇ $(- \ 2 \ 1) \rightarrow 1$

◇ $(- \ 2) \rightarrow -2$

◆ $(* \ n_1 \ n_2 \ \dots \ n_N): n_1 * n_2 * \dots * n_N.$

◇ $(* \ 2 \ 3) \rightarrow 6$

◇ $(* \ 2 \ 3 \ 4) \rightarrow 24$

◆ $(/ \ n_1 \ n_2): n_1 / n_2.$

◇ $(/ \ 1 \ 2) \rightarrow 1/2$

Operacions aritmètiques

- ◆ $(\text{abs } n)$: valor absolut d' n .
 - ◇ $(\text{abs } -2) \rightarrow 2$
- ◆ $(\text{floor } n_1 \ n_2)$: n_1 / n_2 i funció terra.
 - ◇ $(\text{floor } 7.9) \rightarrow 7$; Un argument
 - ◇ $(\text{floor } 7 \ 2) \rightarrow 3$
- ◆ $(\text{ceiling } n_1 \ n_2)$: n_1 / n_2 i funció sostre.
- ◆ $(\text{round } n_1 \ n_2)$: n_1 / n_2 i arrodoneix.
- ◆ $(\text{mod } n_1 \ n_2)$: $n_1 \bmod n_2$.
 - ◇ $(\text{mod } 7 \ 2) \rightarrow 1$
- ◆ $(\text{rem } n_1 \ n_2)$: residu de n_1 / n_2 .
- ◆ $(\text{max } n_1 \ n_2 \ \dots \ n_N)$: avalua al n_i més gros.
 - ◇ $(\text{max } 7 \ 2 \ 3) \rightarrow 7$
- ◆ $(\text{min } n_1 \ n_2 \ \dots \ n_N)$: avalua al n_i més petit.

Operacions relacionals

◆ $(= n_1 \dots n_N): n_1 = \dots = n_N.$

◇ $(= 2 \ 2) \rightarrow t$

◇ $(= 1 \ 1 \ 1) \rightarrow t$

◆ $(/= n_1 \dots n_N):$ avalua a t si tots els n_i son distints.

◇ $(/= 2 \ 2) \rightarrow \text{nil}$

◇ $(/= 1 \ 2 \ 3) \rightarrow t$

◆ $(>= n_1 \dots n_N): n_1 \geq \dots \geq n_N.$

◇ $(>= 4 \ 2 \ 1) \rightarrow t$

◇ $(>= 4 \ 2 \ 3) \rightarrow \text{nil}$

◆ $(> n_1 \dots n_N): n_1 > \dots > n_N.$

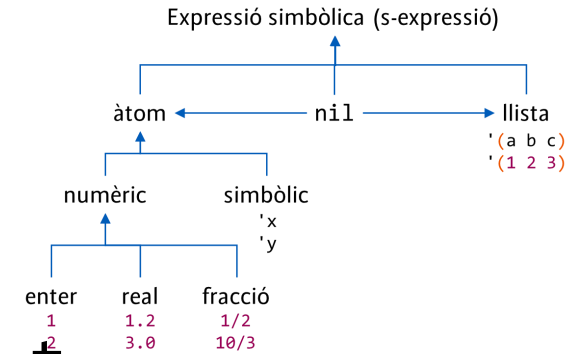
◆ $(<= n_1 \dots n_N): n_1 \leq \dots \leq n_N.$

◆ $(< n_1 \dots n_N): n_1 < \dots < n_N.$

Operacions relacionals

- ◆ `(zerop  $n$ )`: avalua a `t` quan $n = 0$.
 - ◇ `(zerop 2)` → `nil`
 - ◇ `(zerop 0)` → `t`
- ◆ `(plusp  $n$ )`: avalua a `t` quan $n > 0$.
 - ◇ `(plusp -2)` → `nil`
- ◆ `(minusp  $n$ )`: avalua a `t` quan $n < 0$.
 - ◇ `(minusp -2)` → `t`
- ◆ `(evenp  $n$ )`: avalua a `t` quan n és parell.
 - ◇ `(evenp 22)` → `t`
- ◆ `(oddp  $n$ )`: avalua a `t` quan n és senar.
 - ◇ `(oddp 22)` → `nil`

Predicats



◆ $(\text{null } s)$: avalua a t quan s és nil.

◇ $(\text{null nil}) \rightarrow t$ $(\text{null '()}) \rightarrow t$

◆ $(\text{atom } s)$: avalua a t quan s és un àtom.

◇ $(\text{atom 'a}) \rightarrow t$ $(\text{atom '(a)}) \rightarrow \text{nil}$

◆ $(\text{symbolp } s)$: avalua a t quan s és un símbol.

◇ $(\text{symbolp 'a}) \rightarrow t$ $(\text{symbolp 22}) \rightarrow \text{nil}$

◆ $(\text{numberp } s)$: avalua a t quan s és un nombre.

◇ $(\text{numberp 22}) \rightarrow t$ $(\text{numberp 'a}) \rightarrow \text{nil}$

◆ $(\text{listp } s)$: avalua a t quan s és una llista.

◇ $(\text{listp '(a)}) \rightarrow t$ $(\text{listp '()}) \rightarrow t$

Predicats d'igualtat

- ◆ $(eq\ s_1\ s_2)$: avalua a t quan s_1 i s_2 son el mateix símbol (compara apuntadors).

- ◇ $(eq\ 'a\ 'a) \rightarrow t$ $(eq\ '(a)\ '(a)) \rightarrow nil$

- ◆ $(equal\ s_1\ s_2)$: avalua a t quan s_1 i s_2 tenen el mateix valor.

- ◇ $(equal\ 'a\ 'a) \rightarrow t$ $(equal\ '(a)\ '(a)) \rightarrow t$

- ◆ $(= n_1 \dots n_N)$: $n_1 = \dots = n_N$.

- ◇ Només es pot fer servir amb àtoms numèrics.

Operadors lògics

◆ (not s): avalua a t quan s és nil.

◇ (not nil) $\rightarrow$ t (null '()) $\rightarrow$ t

◆ (and $s_1 \dots s_N$): avalua les s_i fins que troba un nil. Avalua al darrer s_i avaluat.

◇ (and t nil t) $\rightarrow$ nil (and 1 2 3) $\rightarrow$ 3

◆ (or $s_1 \dots s_N$): avalua les s_i fins que troba un valor diferent de nil. Avalua al darrer s_i avaluat.

◇ (or nil t nil) $\rightarrow$ t (or 1 2 3) $\rightarrow$ 1

◇ (or nil nil nil) $\rightarrow$ nil

Assignacions

◆ Assignació: SET i SETQ.

◇ `(setq ciutats '(palma inca manacor))`

- SETQ vol dir “set quote”

◇ `(set 'ciutats '(palma inca manacor))`

◆ Van en contra del plantejament funcional.

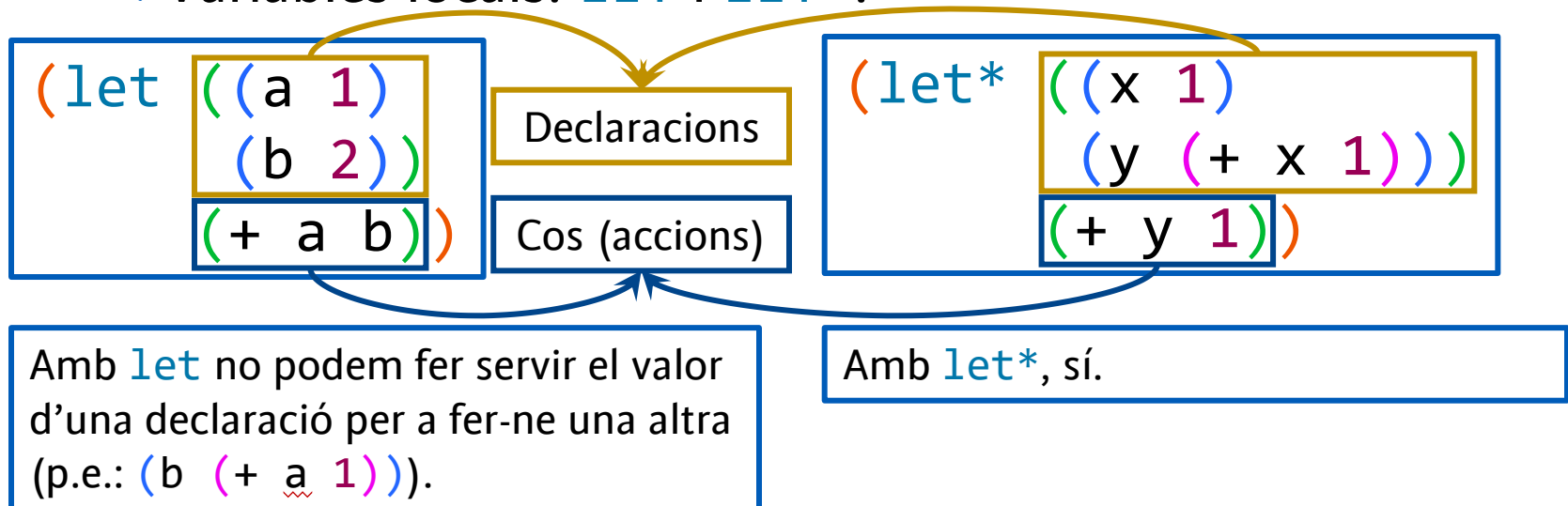
◇ Poden tenir efectes secundaris perillosos

- Assignacions destructives: canviar el valor d'una variable.
- ~~`(setq ciutats (cons 'llucmajor ciutats))`~~
- ~~`(setq total (+ total 1))`~~

◆ En aquesta assignatura, només es permeten per motius clars d'eficiència.

Vinculació de variables: LET i LET*

- ◆ SET i SETQ modifiquen el valor d'una variable, però no l'hem declarada!
 - ◇ És una variable local? És global? Quin àmbit (T4.1)?
- ◆ Declaracions: n'explicitam l'àmbit.
 - ◇ Paràmetres formals de funcions: ja en xerrarem.
 - ◇ Variables locals: **LET** i **LET***.



Diferència EQUAL i EQ

◆ `(set 'l1 '(a b c))` → `(a b c)`

◆ `(set 'l2 l1)` → `(a b c)`

◆ `(set 'l3 '(a b c))` → `(a b c)`

◆ `(eq l1 l2)` → `t` `(equal l1 l2)` → `t`

◆ `(eq l1 l3)` → `nil` `(equal l1 l3)` → `t`

◆ `(eq l2 l3)` → `nil` `(equal l2 l3)` → `t`

◆ `eq`: compara apuntadors (= de Java)

◆ `equal`: compara valors (equals de Java)

Provem l'interpret de LISP

- ◆ Si voleu guardar un registre del que feis:
 - ◇ Menú File > Dribble.
 - ◇ O també `(dribble "nom_fitxer.txt")`
- ◆ Si voleu carregar un fitxer amb codi lisp:
 - ◇ Menú File > Open/load....
 - ◇ O també `(load 'codi.lsp)`
 - `(load "../Documents/codi.lsp")`
 - ◇ Es pot obviar l'extensió ".lsp".
- ◆ Cerca els fitxers a partir del directori on es troba l'interpret.
 - ◇ No els tengueu dins el OneDrive, s'embulla!

Operacions aritmètiques

◆ Amb l'interpret de LISP calculau:

◆ $553 + 47 = 600$

◆ $(+ \text{ 553 47 })$

Operacions aritmètiques

◆ Amb l'interpret de LISP calculau:

◆ $386 - 70 + 38 = 354$

◆ $(+ (- 386 70) 38)$

Operacions aritmètiques

◆ Amb l'interpret de LISP calculau:

◆ $4^3 = 64$

◆ $(* \ 4 \ (* \ 4 \ 4))$

◆ També: $(* \ 4 \ 4 \ 4)$

Operacions aritmètiques

◆ Amb l'interpret de LISP calculau:

◆ $1024 + 2^3 - \left(\frac{27}{3} + 1\right) = 1022$

◆ $(+ \ 1024 \ (* \ 2 \ (* \ 2 \ 2)) \ (- \ (+ \ (/ \ 27 \ 3) \ 1))$

◆ $(- \ (+ \ (* \ (* \ 2 \ 2) \ 2) \ 1024) \ (+ \ (/ \ 27 \ 3) \ 1))$

Manipulació de llistes

- ◆ Les operacions bàsiques són el CAR i el CDR.

- ◇ CAR avalua al primer element d'una llista.

- ◇ CDR avalua a la resta de la llista (sense el primer element).

- ◆ $(\text{car } '(a\ b\ c\ d)) \rightarrow a$

- ◇ (pot ser un àtom o una llista)

- ◆ $(\text{cdr } '(a\ b\ c\ d)) \rightarrow (b\ c\ d)$

- ◇ (sempre és una llista)

- ◆ Quin és el car i el cdr d'una llista buida?

- ◇ nil

Manipulació de llistes

◆ Obteniu el segon element de la llista (a b c d): b

◇ (car (cdr '(a b c d)))

◆ Obteniu el tercer element de la llista (a b c d): c

◇ (car (cdr (cdr '(a b c d))))

Manipulació de llistes

◆ Podem trobar funcions amb el format CxxxR.

◆ x poden ser a o d

◆ això compon els CAR i CDR segons les xxx

◆ `cadr` $\equiv$ `(car (cdr 1))`

◆ `cddr` $\equiv$ `(cdr (cdr 1))`

◆ `caddr` $\equiv$ `(car (cdr (cdr 1)))`

◆ ...

Manipulació de llistes

◆ Altres funcions són:

- ◆ $(\text{null } L)$: avalua a t si L és buida.
- ◆ $(\text{append } L_1 \dots L_N)$: avalua a una llista amb les llistes L_i concatenades.
- ◆ $(\text{member } e \ L)$: avalua a t si l'element e pertany a la llista t .
- ◆ $(\text{remove } e \ L)$: avalua a una llista que és la llista L sense cap aparició de l'element e .
- ◆ $(\text{reverse } L)$: avalua a una llista que és la llista L invertida.

Manipulació de llistes

◆ Obteniu la llista (a b c d) sense l'element *b*: (a c d)

◇ (remove 'b ' (a b c d))

Manipulació de llistes

- ◆ Obteniu una llista amb les llistes (a b) i (a b c d) afegides: (a b a b c d)

- ◇ (append '(a b) '(a b c d))

Manipulació de llistes

◆ Obteniu una llista amb l'element a al principi de la llista $(b\ c\ d)$: $(a\ b\ c\ d)$

◆ `(append '(a) '(b c d))`

Construcció de llistes

◆ Per construir llistes: CONS i LIST

◆ CONS crea una llista amb un element com a *car* i una llista com a *cdr*:

- ◇ `(cons 'a '(b c d))` → `(a b c d)`
- ◇ `(cons '(1 2) '(b c d))` → `((1 2) b c d)`
- ◇ El segon paràmetre hauria de ser una llista.
 - Si obteniu `(a . b)`, no és una llista, sinó un *pair*.

◆ LIST crea una llista a partir del seus arguments:

- ◇ `(list 'a 'b 'c 'd)` → `(a b c d)`
- ◇ `'(a b c d)` → `(a b c d)`

◆ APPEND crea una llista concatenant els paràmetres:

- ◇ `(append '(a b) '(c d))` → `(a b c d)`

Construcció de llistes

- ◆ Creau una llista amb l'element *a* a la segona posició de la llista (b c d): (b a c d)

- ◆ Podem partir de `(setq l '(b c d))`

```
(cons (car l)  
      (cons 'a (cdr l)))
```

```
(append (list (car l) 'a)  
        (cdr l))
```

Definició de noves funcions

- ◆ Per definir noves funcions s'utilitza **defun**:

```
(defun nomfunc (paràmetres)  
  "documentació opcional"  
  expressió)
```

- ◆ Per exemple:

- ◆ (defun suma1 (x) (+ x 1))

- ◆ (suma1 4) → 5

Definició de noves funcions

- ◆ Definiu la funció *quadrat*, que avalua al quadrat d'un nombre:

◇ `(quadrat 4)` → 16

```
(defun quadrat (n) (* n n))
```


Definició de noves funcions

- ◆ Definiu la funció *suma3*, que avalua a la suma de tres nombres:

◇ $(\text{suma3 } 4 \ 5 \ 5) \rightarrow 14$

```
(defun suma3 (x1 x2 x3)
  (+ x1 (+ x2 x3)))
```

Definició de noves funcions

- ◆ Definiu la funció *tercer*, que avalua al tercer element d'una llista:

◇ `(tercer '(a b c))` → c

```
(defun tercer (l)
  (car (cdr (cdr l))))
```

Definició de noves funcions

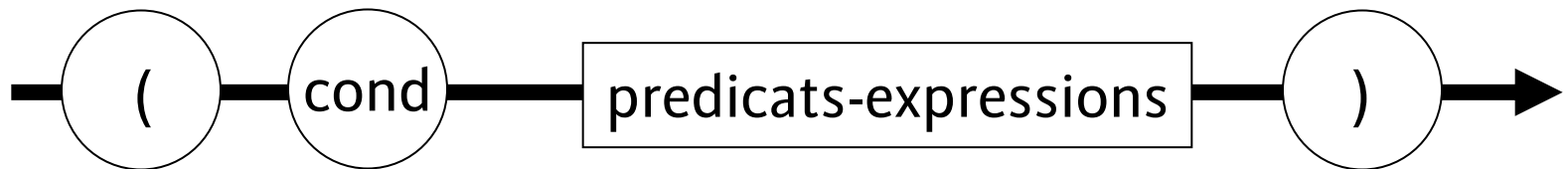
- ◆ Definiu la funció *darrer*, que avalua al darrer element d'una llista:

◇ $(\text{darrer } '(a\ b\ c\ d)) \rightarrow d$

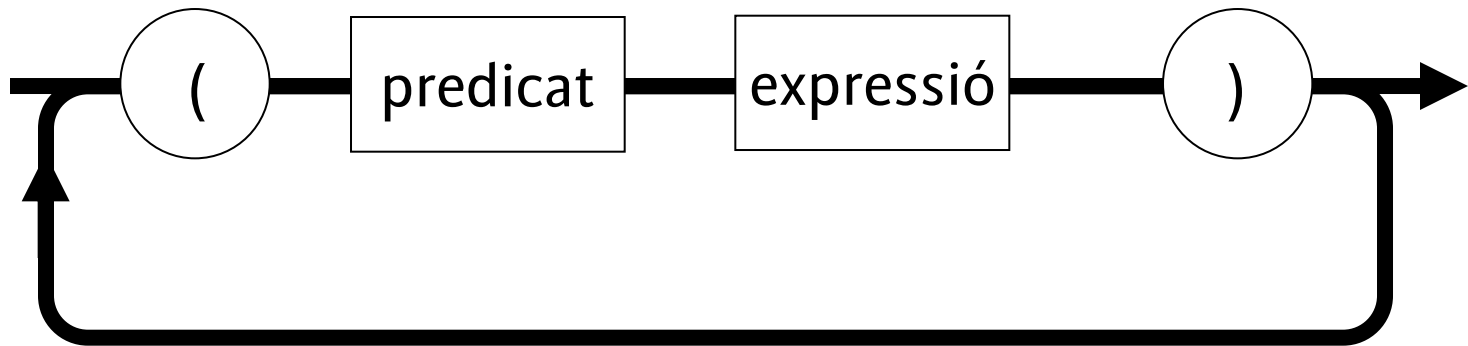
```
(defun darrer (l)
  (car (reverse l)))
```

Expressions conditionals

Cond



Predicats-expressions



Expressions conditionals

```
(cond (predicat-1 expressió-1)
      (predicat-2 expressió-2)
      ...
      (predicat-n expressió-n))
```

- ◇ **Predicat**: condició que s'ha de complir.
 - `(>= n 0)` `(oddp n)` `(null 1)` `t`
- ◇ **Expressió**: expressió que s'avaluarà si es compleix.
 - `0` `'a` `(+ n 1)` `(cons 'a 1)`
- ◇ S'avaluen els predicats fins que se'n troba un de cert.
 - S'avalua la seva expressió, que és al que avalua cond.
 - Predicat “t”: es complirà sempre. Fa d'*else*.
- ◇ Si no es troba cap predicat cert, cond avalua a `nil`.

Definició de noves funcions

◆ Definiu la funció *fesparell*:

$$\diamond \text{fesparell}(n) = \begin{cases} n + 1, & \text{si } n \text{ és senar} \\ n, & \text{si no} \end{cases}$$

$$\diamond (\text{fesparell } 1) \rightarrow 2$$

$$\diamond (\text{fesparell } 2) \rightarrow 2$$

```
(defun fesparell (n)
  (cond ((oddp n) (+ n 1))
        (t n)))
```

Definició de noves funcions

◆ Definiu la funció *absolut*:

$$\diamond \text{absolut}(n) = \begin{cases} n, & \text{si } n \geq 0 \\ -n, & \text{si no} \end{cases}$$

$$\diamond (\text{absolut } 4) \rightarrow 4$$

$$\diamond (\text{absolut } -4) \rightarrow 4$$

```
(defun absolut (n)
  (cond ((>= n 0) n)
        (t (- n))))
```

Definició de noves funcions

- ◆ Definiu la funció *darrer*, que avalua al darrer element d'una llista (sense fer servir *reverse*):

◆ (darrer '(a b c d)) → d

Recursivitat

- ◆ L'estructura per fer iteracions en LISP és la recursivitat:

- ◆ (darrer '(a b c d))

- ◆ (darrer '(b c d))

- ◆ (darrer '(c d))

- ◆ (darrer '(d))

- ◆ (car '(d))

- ◆ d

Recursivitat

◆ Exemple: càlcul de la *longitud* d'una llista.

◇ `(long '(a b c))` → 3

```
(defun long (l)
  (cond ((null l) 0)
        (t (+ 1 (long (cdr l))))))
```

Recursivitat

- ◆ Definiu la funció *darrer*, que avalua al darrer element d'una llista (sense fer servir *reverse*):

◇ $(\text{darrer } '(a\ b\ c\ d)) \rightarrow d$

```
(defun darrer (L)
  (cond ((null (cdr L)) (car L))
        (t (darrer (cdr L)))))
```

Recursivitat

◆ Funció *pertany* que faci el mateix que *member*:

- ◆ `(pertany 'a '(a b c)) → t`
- ◆ `(pertany 'b '(a b c)) → t`
- ◆ `(pertany 'c '(a b c)) → t`
- ◆ `(pertany 'd '(a b c)) → nil`

```
(defun pertany (x l)
  (cond ((null l) nil)
        ((equal x (car l)) t)
        (t (pertany x (cdr l)))))
```

Recursivitat

◆ Definiu la funció exponenciació (*exp*):

◇ $(\text{exp } 2 \ 3) \rightarrow 8$

```
(defun exp (m n)
  (cond ((= n 0) 1)
        (t (* m (exp m (- n 1))))))
```

Recursivitat

- ◆ Definiu la funció *fib* que retorna l'enèsim valor de la sèrie de Fibonacci.

$$fib(n) = \begin{cases} 0, & \text{si } n = 0; \\ 1, & \text{si } n = 1; \\ fib(n-1) + fib(n-2) & \text{altrament.} \end{cases}$$

◇ (fib 6) → 8

```
(defun fib (n)
  (cond ((= n 0) 0)
        ((= n 1) 1)
        (t (+ (fib (- n 1))
                (fib (- n 2))))))
```

Recursivitat

◆ Definiu la funció *dividir*, que fa la divisió entera d'M entre N (tots dos positius).

◆ Quantes vegades està contingut N dins M?

◆ `(dividir 9 2) → 4`

```
(defun dividir (m n)
  (cond ((< m n) 0)
        (t (+ 1 (dividir (- m n) n)))))
```

Recursivitat

- ◆ Definiu la funció *parell*, que fa el mateix que *evenp*, a partir de *dividir*.

- ◇ `(parell 9) → nil`

- ◇ `(parell 6) → t`

```
(defun parell (n)
  (cond ((= (* 2 (dividir n 2)) n) t)
        (t nil)))
```

- ◇ O també:

```
(defun parell (n)
  (= (* 2 (dividir n 2)) n))
```


Recursivitat – llistes

- ◆ Definiu la funció recursiva *senars* que, donada una llista de nombres, en retorna una altra amb tots els nombres parells eliminats.

◇ `(senars '(3 1 8 7 4 10))` → `(3 1 7)`

```
(defun senars (l)
  (cond ((null l) nil)
        ((not (parell (car l)))
         (cons (car l) (senars (cdr l))))
        (t (senars (cdr l)))))
```

Recursivitat – llistes

- ◆ Definiu la funció *esborra* per eliminar la primera aparició d'un element dins una llista.

◇ `(esborra 'a '(1 2 a 3 a b c))` → `(1 2 3 a b c)`

```
(defun esborra (x l)
  (cond ((null l) nil)
        ((equal x (car l)) (cdr l))
        (t (cons (car l)
                  (esborra x (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu la funció *esborra-tot* que esborri totes les aparicions d'un element dins una llista.

◇ `(esborra-tot 'a '(1 2 a 3 a b c))` → `(1 2 3 b c)`

```
(defun esborra-tot (x l)
  (cond ((null l) nil)
        ((equal x (car l))
         (esborra-tot x (cdr l)))
        (t (cons (car l)
                   (esborra-tot x (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu la funció *rdc* que retorna tots els elements d'una llista excepte el darrer.

◇ $(\text{rdc } '(a\ b\ c)) \rightarrow (a\ b)$

```
(defun rdc (l)
  (cond ((null (cdr l)) nil)
        (t (cons (car l) (rdc (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu la funció *snoc* que afegeix un element al final d'una llista (amb *cons*):

◇ $(\text{snoc } 'a \text{ '}(b \text{ } c)) \rightarrow (b \text{ } c \text{ } a)$

```
(defun snoc (x l)
  (cond ((null l) (cons x nil))
        (t (cons (car l) (snoc x (cdr l))))))
```

```
(defun snoc (x l)
  (cond ((null l) (list x))
        (t (cons (car l) (snoc x (cdr l))))))
```

```
(defun snoc (x l)
  (cond ((null (cdr l)) (list (car l) x))
        (t (cons (car l) (snoc x (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu la funció *escala* per multiplicar tots els elements d'una llista per un nombre.

◇ `(escala 2 '(3 5 6))` → `(6 10 12)`

```
(defun escala (x l)
  (cond ((null l) nil)
        (t (cons (* x (car l))
                   (escala x (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu les funcions *màxim* i *mínim* d'una llista de nombres.

◇ `(maxim '(4 1 3 8 5))` → 8

◇ `(minim '(4 1 3 8 5))` → 1

```
(defun minim (l)
  (cond ((null (cdr l)) (car l))
        ((< (car l) (minim (cdr l))) (car l))
        (t (minim (cdr l)))))
```

```
(defun minim2 (l)
  (cond ((null (cdr l)) (car l))
        ((< (car l) (cadr l))
         (minim2 (cons (car l) (cddr l))))
        (t (minim2 (cdr l)))))
```

```
(defun minim (l)
  (cond ((null (cdr l)) (car l))
        ((< (car l) (minim (cdr l))) (car l))
        (t (minim (cdr l)))))
```

Si t, s'avalua dues vegades

```
(defun minimlet (l)
  (cond ((null (cdr l)) (car l))
        (t (let ((mincdr (minimlet (cdr l))))
              (cond ((< (car l) mincdr) (car l))
                    (t mincdr))))))
```

let per evitar-ho

```
(defun min (a b)
  (cond ((< a b) a)
        (t b)))
```

```
(defun minimfun (l)
  (cond ((null (cdr l)) (car l))
        (t (min (car l) (minimfun (cdr l))))))
```

Feim servir una funció per evitar-ho

Recursivitat – llistes

- ◆ Definiu una funció per ordenar una llista de nombres amb el mètode de selecció directa (trobar el mínim a cada passa i moure'l al principi)

◇ `(ordena '(4 1 3 8 5))` → `(1 3 4 5 8)`

```
(defun ordena (l)
  (cond ((null l) nil)
        (t (cons (minim l)
                  (ordena (esborra (minim l) l))))))
```

(esborra x l)
(minim l)

```
(defun ordena (l)
  (cond ((null l) nil)
        (t (let ((mespetit (minim l)))
              (cons mespetit (ordena (esborra mespetit l))))))
```

let per evitar calcular el mínim dues vegades

Recursivitat – llistes

◆ Definiu la funció que *inverteix* una llista.

◇ `(inverteix '(a e i o u))` → `(u o i e a)`

```
(defun inverteix (l)
  (cond ((null l) nil)
        (t (append (inverteix (cdr l))
                     (list (car l))))))
```

```
(defun inverteix2 (l)
  (cond ((null l) nil)
        (t (snoc (car l)
                  (inverteix2 (cdr l))))))
```

```
(defun inverteix3 (l)
  (cond ((null l) l)
        (t (cons (darrer l)
                  (inverteix3 (rdc l))))))
```

La mateixa idea,
però just al revés

Recursivitat – llistes

- ◆ Definiu una funció que *esborra* l'enèsim element d'una llista.

◇ (esborrap 3 '(a e i o u)) → (a e o u)

```
(defun esborrap (n l)
  (cond ((= n 1) (cdr l))
        (t (cons (car l)
                   (esborrap (- n 1) (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu una funció que compti el nombre de *vegades* que una expressió apareix a una llista

◇ `(vegades 'a '(1 2 a 3 (a b) a))` → 2

◇ `(vegades 'x '(s t (x) 3))` → 0

```
(defun vegades (x l)
  (cond ((null l) 0)
        ((equal x (car l))
         (+ 1 (vegades x (cdr l))))
        (t (vegades x (cdr l)))))
```

Recursivitat – llistes

- ◆ Definiu una funció que compti el nombre total d'*àtoms* que hi ha dins una llista

◇ `(atoms '(a (b (c d) e) f g (h i)))` → 9

```
(defun atoms (l)
  (cond ((null l) 0)
        ((listp (car l))
         (+ (atoms (car l)) (atoms (cdr l))))
        (t (+ 1 (atoms (cdr l))))))
```

Recursivitat – llistes

◆ Definiu una funció que *aplana* una llista

- ◇ `(aplana '(a (b (c d) e) f g (h i)))`
 - $\rightarrow$ `(a b c d e f g h i)`

```
(defun aplana (l)
  (cond ((null l) l)
        ((atom (car l))
         (cons (car l) (aplana (cdr l))))
        (t (append (aplana (car l))
                     (aplana (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu una funció que donada una llista retorni la mateixa llista sense els n primers elements

◇ `(treuprimers 3 '(a e i o u))` → `(o u)`

```
(defun treuprimers (n l)
  (cond ((= n 0) l)
        (t (treuprimers (- n 1) (cdr l)))))
```

Recursivitat – llistes

- ◆ Definiu una funció que donada una llista retorni els n primers elements

◇ `(tornaprimers 3 '(a e i o u))` → `(a e i)`

```
(defun tornaprimers (n l)
  (cond ((= n 0) nil)
        (t (cons (car l)
                  (tornaprimers (- n 1) (cdr l))))))
```


Recursivitat – llistes

- ◆ Definiu una funció per inserir un element a la posició enèsima d'una llista

◇ `(insereix 'c 3 '(a b d))` → `(a b c d)`

```
(defun insereix (que on l)
  (cond ((= on 1) (cons que l))
        (t (cons (car l)
                  (insereix que
                           (- on 1)
                           (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu una funció per canviar l'enèsim element d'una llista per l'element donat

◇ `(canvia 3 '(a e i o u) 'x) → (a e x o u)`

```
(defun canvia (on l per)
  (cond ((= on 1) (cons per (cdr l)))
        (t (cons (car l)
                  (canvia (- on 1)
                          (cdr l)
                          per))))))
```

Recursivitat – llistes

- ◆ Definiu *insereix-esquerra* que insereix, a l'esquerra d'un element, un element

- ◇ `(insereix-esquerra 'i 'x '(a e i o u))`
 - $\rightarrow$ `(a e x i o u)`

```
(defun insereix-esquerra (dequi que l)
  (cond ((null l) nil)
        ((equal (car l) dequi) (cons que l))
        (t (cons (car l)
                  (insereix-esquerra dequi
                                     que
                                     (cdr l))))))
```

Recursivitat – llistes

- ◆ Definiu *insereix-dreta* que insereix, a la dreta d'un element, un element

- ◇ `(insereix-dreta 'i 'x '(a e i o u))`
 - $\rightarrow$ `(a e i x o u)`

```
(defun insereix-dreta (dequi que l)
  (cond ((null l) nil)
        ((equal (car l) dequi)
         (cons (car l) (cons que (cdr l)))))
  (t (cons (car l)
            (insereix-dreta dequi
                            que
                            (cdr l))))))
```

Conjunts

◆ Donats dos conjunts (representats per llistes d'elements), escriu les funcions

- ◆ (conjunt-correcte '(a b c d)) $\rightarrow$ t
- ◆ (conjunt-correcte '(a b c c d a)) $\rightarrow$ nil
- ◆ (fer-conjunt '(a b c a d b)) $\rightarrow$ (c a d b)
- ◆ (unio '(a b c) '(a d e c)) $\rightarrow$ (b a d e c)
- ◆ (interseccio '(a b c) '(a d e c)) $\rightarrow$ (a c)
- ◆ (diferencia '(a b c) '(a d e c)) $\rightarrow$ (b)
- ◆ (diferencia-simetrica '(a b c) '(a d e c)) $\rightarrow$ '(b d e)
- ◆ (producte-cartesia '(a b) '(d e))
 - $\rightarrow$ ((a d) (a e) (b d) (b e))

Conjunts

Conjunt correcte

◆ (conjunt-correcte '(a b c d)) → t

◆ (conjunt-correcte '(a b c c d a)) → nil

```
(defun conjunt-correcte (l)
  (cond ((null l) t)
        ((pertany (car l) (cdr l)) nil)
        (t (conjunt-correcte (cdr l)))))
```

Conjunts

Fer conjunt

◆ $(\text{fer-conjunt } '(a\ b\ c\ a\ d\ b)) \rightarrow (c\ a\ d\ b)$

```
(defun fer-conjunt (l)
  (cond ((null l) nil)
        ((pertany (car l) (cdr l))
         (fer-conjunt (cdr l)))
        (t (cons (car l) (fer-conjunt (cdr l))))))
```

Conjunts

Unió

◆ $(\text{unio } '(a\ b\ c)\ '(a\ d\ e\ c)) \rightarrow (b\ a\ d\ e\ c)$

```
(defun unio (c1 c2)
  (cond ((null c1) c2)
        ((pertany (car c1) c2)
         (unio (cdr c1) c2))
        (t (cons (car c1) (unio (cdr c1) c2)))))
```


Conjunts

Intersecció

◆ $(\text{interseccio } '(a\ b\ c)\ '(a\ d\ e\ c)) \rightarrow (a\ c)$

```
(defun interseccio (c1 c2)
  (cond ((null c1) nil)
        ((pertany (car c1) c2)
         (cons (car c1)
                (interseccio (cdr c1) c2)))
        (t (interseccio (cdr c1) c2))))
```

Conjunts

Diferència

◆ $(\text{diferencia } '(a\ b\ c)\ '(a\ d\ e\ c)) \rightarrow (b)$

◇ $(C_1 - C_2)$

```
(defun diferencia (c1 c2)
  (cond ((null c1) nil)
        ((pertany (car c1) c2)
         (diferencia (cdr c1) c2))
        (t (cons (car c1)
                   (diferencia (cdr c1) c2)))))
```

Conjunts

Diferència simètrica

◆ (diferencia-simetrica '(a b c) '(a d e c)) → '(b d e)

◆ $(C_1 - C_2) \cup (C_2 - C_1)$

◆ $(C_1 \cup C_2) - (C_1 \cap C_2)$

```
(defun diferencia-simetrica (c1 c2)
  (unio (diferencia c1 c2)
        (diferencia c2 c1)))
```

Conjunts

Producte cartesià

◆ (producte-cartesia '(a b) '(d e))

◆ → ((a d) (a e) (b d) (b e))

```
(defun producte-cartesia (c1 c2)
  (cond ((null c1) nil)
        (t (append (fer-llistes (car c1) c2)
                     (producte-cartesia (cdr c1)
                                         c2))))))
```

◆ (fer-llistes 'a '(d e)) → ((a d) (a e))

```
(defun fer-llistes (a l)
  (cond ((null l) nil)
        (t (cons (list a (car l))
                   (fer-llistes a (cdr l))))))
```

Entrada i sortida

Expressions pel terminal

◆ Lectura d'una expressió, avalua a l'expressió:

◇ (read)

- Llegeix una expressió, no (només) text.

◆ Escriptura d'una expressió (avaluen a l'expressió):

◇ (prin1 expressió)

- Imprimeix segons un format acceptable per a *read*.

◇ (princ expressió)

- Imprimeix sense quotes.

◇ (terpri)

- Imprimeix un salt de línia.

◇ (print expressió)

- Imprimeix amb *prin1* seguit d'un salt de línia.

◇ (pprint expressió)

- Imprimeix amb *prin1* “pretty”.

Entrada i sortida

Fitxers de text

◆ Obertura d'un fitxer de text:

◇ (open nom-fitxer)

- Obre *nom-fitxer* i avalua a l'apuntador del fitxer obert.
- Accepta *keywords*:
 - **:direction** – permís de lectura o escriptura
 - **:input** – només de lectura (per defecte)
 - **:output** – només d'escriptura
 - **:probe** – comprova si existeix<
 - **:if-exists** – si és **:output**, què fer si existeix?
 - **:rename** – conserva un .bak (per defecte)
 - **:append** – afegeix al final
 - **:supersede** – substitueix el fitxer
 - **:if-does-not-exist** – què fer si no existeix?
 - **:error** – missatge d'error (per defecte en **:input**)
 - **:create** – crea el fitxer (per defecte en **:output**)
- Per exemple: (open "fitxer.txt" **:direction** **:input**)

Entrada i sortida – fitxers de text

Lectura d'expressions i caràcters

◆ Lectura d'una expressió, avalua a l'expressió:

◇ `(read fp eofp eof)`

- Llegeix una expressió, no text.
- Arguments:
 - `fp`: apuntador del fitxer obert amb *open*.
 - `eofp`: si és `t`, ocorrerà un error en arribar a l'*EOF* (per defecte, `t`).
 - `eof`: valor a retornar en arribar a l'*EOF* (per defecte, `nil`).
- Per exemple: `(read fp nil nil)`

◆ Lectura d'un caràcter, avalua al caràcter:

◇ `(read-char fp eofp eof)`

- Llegeix un caràcter.
- Arguments: com abans.

Entrada i sortida – fitxers de text

Escriptura d'expressions i caràcters, tancament

◆ Escriptura d'una expressió:

◇ `(prin1 expressió fp)`

- Imprimeix segons un format acceptable per a *read*.
 - `fp`: apuntador del fitxer obert amb *open*.

◇ I la resta de funcions descrites abans, similarment.

◆ Escriptura d'un caràcter:

◇ `(write-char caràcter fp)`

◆ Tancament d'un fitxer:

◇ `(close fp)`

Entrada i sortida – fitxers de text – exemples

Lectura d'una expressió

◆ Llegeix l'expressió d'un fitxer:

- ◇ `(cadr (llegeix-exp "expressio.txt"))`
 - $\rightarrow (d\ e\ f)$

```
(defun llegeix-exp (nom)
  (let* ((fp (open nom))
        (e (read fp nil nil)))
    (close fp)
    e))
```

```
((a b c)
 (d e f)
 (g h i))
```

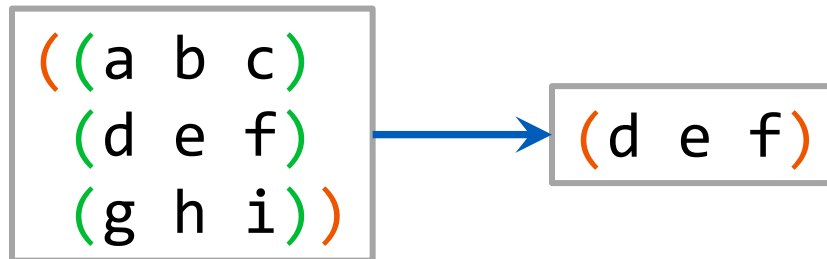
Entrada i sortida – fitxers de text – exemples

Escriptura d'una expressió

◆ Escriu una expressió a un fitxer:

◇ `(escriu-exp "expressio2.txt"`
 `(cadr (llegeix-exp "expressio.txt")))`

```
(defun escriu-exp (nom e)
  (let ((fp (open nom :direction :output)))
    (print e fp)
    (close fp)))
```



Entrada i sortida – fitxers de text – exemples

Lectura de caràcters

◆ Llegeix caràcters d'un fitxer i posa'ls a una llista:

◇ (`llegeix "text.txt"`)

- → (`#\h #\o #\l #\a #\Space #\c #\o #\m #\Space #\Newline #\Space #\Space #\v #\a #\Space #\a #\i #\x #\o #\.` `#\Newline`)

```
(defun llegeix (nom)
  (let* ((fp (open nom))
        (contingut (llegeix-intern fp)))
    (close fp)
    contingut))
```

```
(defun llegeix-intern (fp)
  (let ((c (read-char fp nil nil)))
    (cond ((null c) '())
          (t (cons c (llegeix-intern fp))))))
```

hola com
va això.

Entrada i sortida – fitxers de text – exemples

Escriptura de caràcters

◆ Escriu una llista de caràcters en un fitxer:

◇ `(escriu "text2.txt" '(#\h #\o #\l #\a #\Space #\c
#\o #\m #\Space #\Newline #\Space #\Space #\v #\a
#\Space #\a #\i #\x #\o #\ . #\Newline))`

```
(defun escriu (nom contingut)
  (let ((fp (open nom :direction :output)))
    (escriu-intern fp contingut)
    (close fp)))
```

hola com
va això.

```
(defun escriu-intern (fp contingut)
  (cond ((null contingut) nil)
        (t (write-char (car contingut) fp)
            (escriu-intern fp (cdr contingut))))))
```

Entrada i sortida – fitxers de text – exemples

Esriptura de caràcters

◆ Afegiu una llista de caràcters en un fitxer:

- ◇ `(afegeix "text3.txt" ' (#\h #\o #\l #\a #\Newline))`
- ◇ `(afegeix "text3.txt" ' (#\h #\o #\l #\a #\Newline))`
- ◇ `(afegeix "text3.txt" ' (#\h #\o #\l #\a #\Newline))`

```
(defun afegeix (nom contingut)
  (let ((fp (open nom :direction :output
                  :if-exists :append
                  :if-does-not-exist :create)))
    (escriu-intern fp contingut)
    (close fp)))
```

```
hola
hola
hola
```

La funció EVAL

- ◆Avalua una expressió.

- ◇`(eval '(+ 2 3))` → 5

- ◆És el cor de l'interpret de LISP.

- ◇`(repeteix (eval (read)))`

```
(defun repeteix (x)
  (print x) (princ '>>>>)
  (repeteix (eval (read))))
```

La funció EVAL

- ◆ EVAL pot fer que els seus arguments s'avaluïn dues vegades:
 - ◇ Abans de cridar a la funció EVAL, com a conseqüència del procés habitual d'avaluació.
 - ◇ Com a resultat de la crida a EVAL.

La funció EVAL

◆Exemple:

◆(setq anys 12)

◆(setq mesos 5)

◆(setq info 'edat-en-mesos)

◆(setq edat-en-mesos '(+ (* anys 12) mesos))

◆A quin valor avaluen:

◆(eval info) (+ (* anys 12) mesos)

◆(eval edat-en-mesos) 149

◆(eval (eval info)) 149

La funció EVAL

Avaluació d'un fitxer

◆ Carrega un fitxer:

◇ `(carrega (open "senzill.txt" :direction :input) "Carregant el fitxer")`

```
(defun carrega (fp e)
  (cond ((null e) (close fp))
        (t (print (eval e))
            (carrega fp (read fp nil nil))))))
```

```
(+ 1 2)
(+ 2 3)
(defun suma (a b)
  (+ a b))
(suma 10 2)
```



```
"Carregant el fitxer"
3
5
suma
12
t
```

Llistes de propietats (plists)

- ◆ Cada àtom té una llista de propietats associada
- ◆ A més de les propietats predefinides (nom, valor...) podem afegir, eliminar i consultar propietats amb les funcions *putprop*, *remprop* i *get*.

Compte amb les assignacions destructives

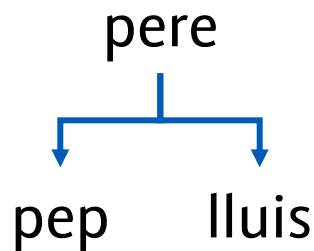
Llistes de propietats

◆ *putprop*: afegeix una propietat a un àtom

◇ (putprop àtom valor propietat)

◇ (putprop 'pep 'pere 'pare)

◇ (putprop 'pere '(pep lluis) 'fills)



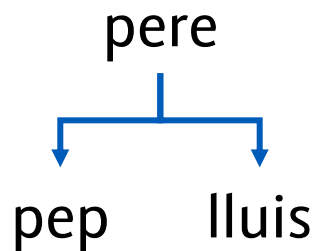
Compte amb les assignacions destructives

Llistes de propietats

◆ *remprop*: elimina una propietat d'un àtom

◇ (remprop àtom propietat)

◇ (remprop 'pep 'pare)



Això és destructiu!

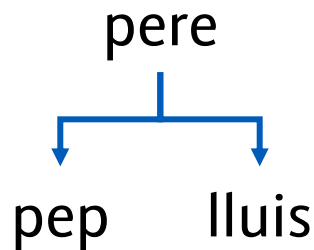
Llistes de propietats

◆ *get*: obté el valor d'una propietat d'un àtom

◇ (get àtom propietat)

◇ (get 'pep 'pare) → pere

◇ (get 'pere 'fills) → (pep lluis)



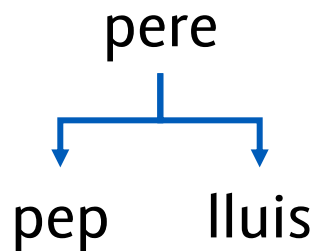
Llistes de propietats

◆ `symbol-plist`: mostra tota la llista de propietats d'un àtom

◆ `(symbol-plist àtom)`

◆ `(symbol-plist 'pere)` → `(fills (pep lluis))`

◆ `(symbol-plist 'pep)` → `(pare pere)`

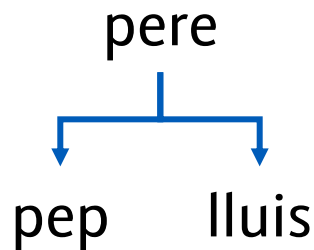


Llistes de propietats

- ◆ Les propietats es poden utilitzar com a mecanisme per emmagatzemar valors dels àtoms

```
(defun germans (x)  
  (remove x (get (get x 'pare) 'fills))))
```

◇ (germans 'pep) → (lluis)



Exemple llista de propietats

- ◆ Imaginau que tenim la funció *color*, que serveix per canviar el color d'escriptura:

- ◇ `(color 255 0 0)`

- ◆ Volem guardar el color de *cercle* dins la propietat *color*.

- ◇ `(putprop 'cercle '(255 0 0) 'color)`

- ◆ Per fer-lo servir...

- ◇ ~~`(color (get 'cercle 'color))`~~

- → ~~`(color '(255 0 0))`~~; volem `(color 255 0 0)`

- ◇ `(eval (cons 'color (get 'cercle 'color)))`

- → `(color 255 0 0)`

- ◇ `(apply 'color (get 'cercle 'color))`

- Ho veurem més endavant.

Estructures complexes

XLISP-PLUS suporta altres estructures (*structs*, *objects*, ...), però és molt fàcil caure'n en l'assignació destructiva!

- ◆ Podem agrupar diferents valors dins una llista:
 - ◇ `'(valor1 valor2...)`
 - ◇ Però hem de recordar quin és el `car`, quin el `cadr`, etc.
- ◆ Podem seguir la mateixa idea que les llistes de propietats, però sense haver-les de relacionar amb un símbol.
 - ◇ `'(clau1 valor1 clau2 valor2 ...)`
- ◆ Haurem de fer un parell de funcions no destructives:
 - ◇ `(pget llista clau)` → obté el valor de `clau`
 - ◇ `(pcons llista clau valor)` → llista amb `clau` i `valor` a l'inici
 - ◇ `(prem llista clau)` → llista sense `clau` (ni el seu valor)
 - ◇ `(pset llista clau valor)` → llista amb `clau` = `valor`
 - ◇ `(pext llista1 llista2)` → llista1, amb els canvis de llista2
- ◆ LISP té llistes associatives, però són un poc estranyes.
 - ◇ `'((clau1 . valor1) (clau2 . valor2) ...)`

Algunes funcions útils del sistema XLISP

◆ (time expr)

- ◇ Mesura el temps d'execució d'una expressió.

◆ (get-internal-real-time)

- ◇ Temps que ha passat des de l'inici de l'interpret.
 - `internal-time-units-per-second`

◆ (get-key)

- ◇ Obté el codi de la tecla pitjada.

◆ (code-char n)

- ◇ Obté el caràcter d'un codi.
- ◇ `(eq (code-char 97) #\a) → t`

◆ (char-code c)

- ◇ Obté el codi d'un caràcter.
- ◇ `(eq 97 (char-code #\a)) → t`

◆ (random n)

- ◇ Nombre aleatori en el rang $[0, n)$.
 - Segons el tipus d' n .
- ◇ `(random 10) → enter [0,10)`
- ◇ `(random 1.0) → real [0,1)`

◆ (make-random-state t)

- ◇ Crea un estat aleatori per a *random*:
 - `(setq rs (make-random-state t))`
 - `(random n rs)`

◆ (break m)

- ◇ Entra en un *break loop* amb missatge *m*.
- ◇ També s'entra en un *break loop* en trobar un error.
- ◇ Podem veure variables de l'entorn.

◆ (backtrace <n>)

- ◇ Mostra *n* nivells de la traça d'error.

◆ (continue)

- ◇ Continua del *break loop*.

Les funcions d'ordre superior

- ◆ Són aquelles que fan servir altres funcions com a arguments.
- ◆ Algunes de les que estan definides són:
 - ◇ APPLY
 - ◇ MAPCAR
 - ◇ REDUCE
 - ◇ SOME
 - ◇ EVERY
 - ◇ FUNCALL
- ◆ També en podem definir nosaltres!

Les funcions d'ordre superior

◆ (apply funció [args...] 1-resta-args)

◇ Pren (com a mínim) dos arguments:

- el primer ha d'avaluar a una funció,
- enmig, opcionalment, hi ha els arguments a passar a la funció,
 - és habitual no fer-los servir
- el darrer és una llista de la resta d'arguments a passar.

◆ Per exemple:

◇ (apply '+ '(1 2)) → 3

- (apply '+ 1 '(2)) → 3
- (apply '+ 1 2 '()) → 3

◇ (apply 'long '((a b c d))) → 4

◇ (apply 'append '((a b) (c d e)))
• → (a b c d e)

Les funcions d'ordre superior

◆ (mapcar funció llista...)

- ◇ Aplica una funció a cada membre d'una llista.
- ◇ Avalua a una llista amb les avaluacions.
- ◇ Accepta més d'una llista d'elements:
 - més arguments a la funció.
- ◇ En altres llenguatges: “map”.

◆ Per exemple:

- ◇ (mapcar 'suma1 '(2 5 1 9)) → (3 6 2 10)
- ◇ (mapcar '+ '(1 2) '(3 4)) → (4 6)

Les funcions d'ordre superior

◆ (reduce fun llist :initial-value :start :end)

◇ Resumeix la llista *llis* a un únic valor amb la funció *fun*.

◇ Arguments *keyword* opcionals:

- **:initial-value** – valor de partida (o 1r valor de *llis*)
- **:start** – índex d'inici (partint de zero)
- **:end** – índex del darrer element considerat (partint de zero)

◇ Molt utilitzat a *Big Data* (esquema *map-reduce*).

◇ Diferència amb (apply fun llist)?

- Funció amb només dos arguments a la vegada.
- Valor inicial.

◆ Per exemple:

◇ (reduce '+ '(1 2 3)) → 6

◇ (reduce '+ (mapcar 'length '("hola" "com" "va")))

- → 9

Les funcions d'ordre superior

◆ (some funció llista...)

- ◇ Aplica una funció als membres de la llista fins que avalua a no-nil. Avalua a aquest valor.
 - Si no troba cap valor no-nil, avalua a nil.
- ◇ Accepta més d'una llista d'elements:
 - més arguments a la funció.
- ◇ Semblant a *(or (funció e1) (funció e2)...) .*
- ◇ També: funció *notany*.

◆ Per exemple:

- ◇ (some 'numberp '(a 1 b)) → t
- ◇ (some 'numberp '(a b c)) → nil
- ◇ (some '> '(5 5 5) '(2 4 6)) → t

Les funcions d'ordre superior

◆ (every funció llista...)

- ◇ Aplica una funció a tots els membres de la llista fins que avalua a nil. Avalua a nil.
 - Si no troba cap valor nil, avalua a t.
- ◇ Accepta més d'una llista d'elements:
 - més arguments a la funció.
- ◇ Semblant a *(and (funció e1) (funció e2)...) .*
- ◇ També: funció *notevery*.

◆ Per exemple:

- ◇ (every 'numberp '(a 1 b)) → nil
- ◇ (every 'numberp '(1 2 3)) → t
- ◇ (every '> '(2 3 4) '(1 2 3)) → t

Les funcions d'ordre superior

- ◆ `(funcall funció arguments...)`
- ◆ Executa la funció amb els arguments indicats.
 - ◇ Semblant a *apply*, que necessita una *llista-arguments*.
- ◆ Per exemple:
 - ◇ `(funcall '+ 1 2) → 3`
 - ◇ `(funcall 'car '(a b c)) → a`
 - ◇ `(funcall 'append '(a b) '(c d)) → (a b c d)`
- ◆ Amb *apply*:
 - ◇ `(apply '+ '(1 2)) → 3`
 - ◇ `(apply 'car '((a b c))) → a`
 - ◇ `(apply 'append '((a b) (c d))) → (a b c d)`

Funcions anònimes

- ◆ Si s'han d'utilitzar funcions molt simples com arguments de funcions d'ordre superior podem fer ús de les funcions anònimes.
- ◆ Són funcions sense nom, que feim servir momentàniament i que no pensam tornar a fer servir.
- ◆ Les funcions anònimes s'anomenen *expressions lambda*.

Funcions anònimes

◆ Per exemple, volem aplicar *suma1* amb *mapcar*.

◇ Fins ara, hem de definir *suma1* a part.

- `(defun suma1 (x) (+ 1 x))`
- `(mapcar 'suma1 '(1 2 3 4)) → (2 3 4 5)`

◆ Podem escriure *suma1* com una funció anònima:

◇ `(lambda (x) (+ 1 x)) → #<Closure: #262a20c>`

◇ Comprovació ràpida amb *funcall*:

- `(funcall (lambda (x) (+ 1 x)) 1) → 2`

◇ I així fer-la servir directament dins *mapcar*:

- `(mapcar (lambda (x) (+ 1 x)) '(1 2 3 4))`
 - `→ (2 3 4 5)`

Les funcions d'ordre superior

- ◆ Defineix la funció *esborra-si* que esborra un element d'una llista si es compleix una determinada condició:

◇ `(esborra-si 'evenp '(1 2 3 4 5 6))` → `(1 3 5)`

```
(defun esborra-si (f l)
  (cond ((null l) nil)
        ((funcall f (car l))
         (esborra-si f (cdr l)))
        (t (cons (car l)
                   (esborra-si f (cdr l))))))
```

Les funcions d'ordre superior

- ◆ Defineix la funció *afegeix-coes* que, donada una llista de llistes, retorna una llista amb els elements de les coes de totes les llistes (els seus *cdr*).

- ◇ `(afegeix-coes '((1 2 3) (4 5 6) (7 8 9)))`
 - $\rightarrow$ (2 3 5 6 8 9)

```
(defun afegeix-coes (l)
  (apply 'append (mapcar 'cdr l)))
```

Les funcions d'ordre superior

◆ Defineix *meu-mapcar* que fa el mateix que *mapcar*.

◇ (amb només una llista)

◇ `(meu-mapcar 'suma1 '(2 5 1 9))` → `(3 6 2 10)`

```
(defun meu-mapcar (f l)
  (cond ((null l) nil)
        (t (cons (funcall f (car l))
                   (meu-mapcar f (cdr l))))))
```

Producte escalar

◆ Funció *pescalar*:

◇ `(pescalar '(1 2 3) '(4 5 6))` → 32

```
(defun pescalar (v1 v2)
  (cond ((null v1) 0)
        (t (+ (* (car v1) (car v2))
                (pescalar (cdr v1) (cdr v2))))))
```

Producte escalar

- ◆ Defineix la funció *pescalar* que fa el producte escalar dels elements de la llista donada.

- ◇ $a \cdot b = \sum_{i=1}^n a_i b_i$

- ◇ (pescalar ' ((1 2 3) (4 5 6))) → 32

- ◆ Passes:

1. Posar els a_i i b_i per parelles.

- (transposta ' ((1 2 3) (4 5 6))) → ((1 4) (2 5) (3 6))

2. Aplicar *mult* a cada element de la llista.

3. Resumir amb + tots els elements de la llista.

Funció transposta

◆ Si tenguéssim dos paràmetres:

- ◇ `(transposta '(1 2 3) '(4 5 6))`
 - $\rightarrow ((1\ 4)\ (2\ 5)\ (3\ 6))$

```
(defun transposta (l1 l2)
  (cond ((null l1) nil)
        (t (cons (list (car l1) (car l2))
                   (transposta (cdr l1) (cdr l2))))))
```

Funció transposta

◆ Però en tenim un:

◇ (transposta '((1 2 3) (4 5 6)))
• → ((1 4) (2 5) (3 6))

```
(defun transposta (l)
  (cond ((null (car l)) nil)
        (t (cons (mapcar 'car l)
                   (transposta (mapcar 'cdr l))))))
```

Funció transposta

◆ Sense recursivitat:

- ◇ `(transposta '((1 2 3) (4 5 6)))`
 - $\rightarrow ((1\ 4)\ (2\ 5)\ (3\ 6))$

```
(defun transposta (l)
  (apply 'mapcar (cons 'list l)))
```

- ◇ Aplica mapcar a la llista d'arguments
`'(list (1 2 3) (4 5 6))`
 - mapcar amb dues llistes: dos arguments a la funció list.

◆ També es pot escriure així:

```
(defun transposta (l)
  (apply 'mapcar 'list l))
```

Producte escalar

- ◆ Defineix la funció *pescalar* que fa el producte escalar dels elements de la llista donada.

- ◇ $a \cdot b = \sum_{i=1}^n a_i b_i$

- ◇ `(pescalar '((1 2 3) (4 5 6)))` → 32

- ◆ Passes:

1. Posar els a_i i b_i per parelles.

- `(transposta '((1 2 3) (4 5 6)))` → `((1 4) (2 5) (3 6))`

2. Aplicar *mult* a cada element de la llista.

- `(mapcar 'mult '((1 4) (2 5) (3 6)))` → `(4 10 18)`

3. Resumir amb + tots els elements de la llista.

Producte escalar

- ◆ L'operador `*` multiplica tots els seus arguments.

- ◇ `(* 2 3) → 6`

- ◆ En aplicar *mapcar*, l'argument serà una llista.

- ◇ `(mult '(2 3)) → 6`

- `(mapcar 'mult '((1 4) (2 5) (3 6))) → (4 10 18)`

```
(defun mult (l)
  (cond ((null l) 1)
        (t (* (car l) (mult (cdr l))))))
```

- ◇ Alternativament: `(apply '* '(2 3)) → 6`

```
(mapcar (lambda (l) (apply '* l)) '((1 4) (2 5) (3 6)))
```

Producte escalar

- ◆ Defineix la funció *pescalar* que fa el producte escalar dels elements de la llista donada.

- ◇ $a \cdot b = \sum_{i=1}^n a_i b_i$

- ◇ `(pescalar '((1 2 3) (4 5 6)))` → 32

- ◆ Passes:

1. Posar els a_i i b_i per parelles.

- `(transposta '((1 2 3) (4 5 6)))` → `((1 4) (2 5) (3 6))`

2. Aplicar *mult* a cada element de la llista.

- `(mapcar 'mult '((1 4) (2 5) (3 6)))` → `(4 10 18)`

3. Resumir amb + tots els elements de la llista.

- `(reduce '+ '(4 10 18))` → 32

Producte escalar

◆ Defineix la funció *pescalar*.

◇ `(pescalar '((1 2 3) (4 5 6)))` → 32

```
(defun transposta (l)
  (cond ((null (car l)) nil)
        (t (cons (mapcar 'car l)
                   (transposta (mapcar 'cdr l))))))

(defun mult (l)
  (cond ((null l) 1)
        (t (* (car l) (mult (cdr l))))))

(defun pescalar (l)
  (reduce '+ (mapcar 'mult (transposta l))))
```

Producte escalar

◆ Defineix la funció *pescalar*.

◇ `(pescalar '((1 2 3) (4 5 6)))` → 32

```
(defun transposta (l)
  (cond ((null (car l)) nil)
        (t (cons (mapcar 'car l)
                   (transposta (mapcar 'cdr l))))))
```

```
(defun mult (l)
  (apply '* l))
```

```
(defun pescalar (l)
  (reduce '+ (mapcar 'mult (transposta l))))
```


Producte escalar

◆ Defineix la funció *pescalar*.

◇ `(pescalar '((1 2 3) (4 5 6)))` → 32

```
(defun transposta (l)
  (apply 'mapcar (cons 'list l)))
```

```
(defun pescalar (l)
  (reduce '+ (mapcar (lambda (l) (apply '* l))
                    (transposta l)))))
```

```
(defun pescalar (l)
  (reduce '+ (mapcar (lambda (l) (apply '* l))
                    (apply 'mapcar
                          (cons 'list l))))))
```

Producte escalar

◆ Encara més simplificat:

1. Posar els a_i i b_i per parelles.

- (transposta '((1 2 3) (4 5 6))) → ((1 4) (2 5) (3 6))

2. Aplicar *mult* a cada element de la llista.

- (mapcar 'mult '((1 4) (2 5) (3 6))) → (4 10 18)

3. Resumir amb + tots els elements de la llista.

- (apply '+ '(4 10 18)) → 32

→ (apply 'mapcar '* 1)

mapcar rep tres paràmetres:

1. '*
2. '(1 2 3)
3. '(4 5 6)

→ (reduce '+ (apply 'mapcar '* 1))

Esquema *map-reduce*!

Conjunts

Producte cartesià

◆ (fer-l·listes 'a '(x y)) → ((a x) (a y))

```
(defun fer-l·listes (a l)
  (mapcar (lambda (b) (list a b)) l))
```

◆ (quasi '(a b c) '(x y))

◇ → (((a x) (a y)) ((b x) (b y)) ((c x) (c y)))

```
(defun quasi (c1 c2)
  (mapcar (lambda (a) (fer-l·listes a c2)) c1))
```

◆ (producte-cartesia '(a b c) '(x y))

◇ → ((a x) (a y) (b x) (b y) (c x) (c y))

```
(defun producte-cartesia (c1 c2)
  (apply 'append (quasi c1 c2)))
```

Conjunts

Producte cartesià

- ◆ (producte-cartesia '(a b c) '(x y))
 - ◇ → ((a x) (a y) (b x) (b y) (c x) (c y))

```
(defun producte-cartesia (c1 c2)
  (apply 'append
    (mapcar (lambda (a)
      (mapcar (lambda (b)
        (list a b))
        c2))
      c1)))
```

(fes-l·listes a c2)

(quasi c1 c2)

Fixau-vos que a la lambda verda, a no és local.
A la lambda porpra, a sí que ho és, però c2 no.
Una lambda fa un *tancament* (*closure*). Ja en xerrarem a teoria.